

第 7 章：Policy Gradient 与 REINFORCE：从 G_t 到 Baseline

7.1 本章符号说明

符号/缩写	English	中文	含义
PG	Policy Gradient	策略梯度	直接对策略参数求梯度、最大化期望回报的一类方法。
REINFORCE	REINFORCE Algorithm	REINFORCE 算法	使用完整轨迹和 Monte Carlo return 的基础 Policy Gradient 算法。
MC	Monte Carlo	蒙特卡洛	等轨迹结束后，用实际采样到的完整 return 做估计。
$S_t / A_t / R_{t+1}$	State / Action / Reward Random Variable	状态 / 动作 / 奖励随机变量	时刻 t 的状态、动作，以及执行动作后收到的奖励。
$\pi_\theta(a s)$	Parameterized Policy	参数化策略	参数为 θ 的动作概率分布。
θ	Policy Parameters	策略参数	Policy network 的可学习参数。
τ	Trajectory	轨迹	一段 state-action-reward 序列。
$p_\theta(\tau)$	Trajectory Probability	轨迹概率	当前策略产生整条轨迹 τ 的概率或概率密度。
$P(s' s,a)$	Transition Probability	状态转移概率	环境从 (s,a) 转移到下一状态 s' 的概率，不依赖策略参数 θ 。
$J(\theta)$	Policy Objective	策略目标函数	当前策略的期望 return，是策略需要最大化的目标。
G_t	Return / Reward-to-go	从时刻 t 开始的回报	从 t 之后实际采样到的累计折扣奖励。
$Q_\pi(s,a)$	Action-value Function	动作价值函数	在 s 先执行 a 、以后遵循 π 时， G_t 的条件期望。
$V_\pi(s)$	State-value Function	状态价值函数	在 s 按策略 π 行动时， G_t 的条件期望。
$b(s)$	Baseline Function	基线函数	从策略更新信号中减去、且不能依赖当前动作的参考值。
$A_\pi(s,a)$	Advantage Function	优势函数	$Q_\pi(s,a) - V_\pi(s)$ ，表示动作相对当前策略平均动作好多少。
$\hat{A}_t^{MC}$	Monte Carlo Advantage Estimate	蒙特卡洛优势估计	用单条样本的 $G_t - b(s_t)$ 估计理论 advantage。帽子表示估计量。
$f_\theta(s)$	Policy Logits	策略 logits	离散策略网络在 softmax 前输出的未归一化分数。
$\mu_\theta(s) / \sigma_\theta(s)$	Gaussian Mean / Standard Deviation	高斯均值 / 标准差	连续动作策略输出的分布参数。

符号/缩写	English	中文	含义
$\mathcal{N}(x; \mu, \sigma)$	Gaussian Probability Density	高斯概率密度	均值为 μ 、标准差为 σ 的高斯分布在 x 处的概率密度；分号左边是被评价的取值，右边是分布参数。
K_t / U_t	Discrete Type / Continuous Parameter	离散类型 / 连续参数随机变量	混合动作 $A_t = (K_t, U_t)$ 的两个组成部分；小写 k_t, u_t 表示采样值。
θ_d / θ_c	Discrete-head / Continuous-head Parameters	离散分支 / 连续分支参数	混合策略中 categorical head 和 continuous head 的参数。
α_π / α_V	Policy / Baseline Learning Rate	策略 / 基线学习率	分别控制策略和可学习 baseline 的更新步长。
L	Loss Function	损失函数	实现中要最小化的训练目标。

7.2 本章目标

学完本章，你应该能够沿着一条完整链路解释：

1. Policy Gradient 为什么会出现 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 。
2. 理论公式为什么写 $Q_{\pi}(s, a)$ ，而 REINFORCE 实现为什么使用采样到的 G_t 。
3. Baseline 是怎样从 control variate（控制变量）思想得到的。
4. 为什么 baseline 不能依赖当前动作，以及它为何不改变梯度期望。
5. 为什么选择 $V_{\pi}(s)$ 作为 baseline 后，会自然出现 advantage。
6. $A_{\pi}(s, a)$ 、 $G_t - V_{\pi}(s_t)$ 和 Actor-Critic 中的 advantage estimate 有什么区别。

7.3 本章主线与边界

本章只解决 **Monte Carlo Policy Gradient** 的问题，推理顺序是：

```

最大化期望 return
-> log-derivative trick
-> Policy Gradient theorem 中出现  $Q_{\pi}(s, a)$ 
-> 用一次采样的  $G_t$  估计  $Q_{\pi}(s, a)$ 
-> 得到 REINFORCE
-> 给高方差的  $G_t$  引入 action-independent baseline
-> 取  $baseline = V_{\pi}(s)$ 
->  $G_t - V_{\pi}(s_t)$  成为 advantage 的 MC 估计

```

本章不会把后续算法提前塞进 REINFORCE：

- 本章的策略学习信号仍来自完整 Monte Carlo return。
- 第 8 章才引入 Critic、bootstrap、TD error、n-step return、GAE、A2C 和 A3C。
- 第 9 章才讨论 PPO 如何限制一次策略更新的幅度。
- Entropy bonus 作为 Actor-Critic 的工程目标组成，也放到第 8 章说明。

7.4 为什么直接优化策略

7.4.1 Value-based 方法留下的问题

Value-based 方法先学习 $Q(s,a)$ ，再通过：

$$a^* = \arg \max_a Q(s,a)$$

选择动作。离散动作较少时，这很自然；但以下场景更适合直接学习 policy：

- 动作连续，难以枚举所有动作再取最大值。
- 需要显式随机策略，例如同一个状态允许多种合理动作。
- 希望直接控制动作分布，而不是先学习 Q 再导出策略。
- $\arg \max_a Q(s,a)$ 本身很难求。

Policy Gradient 直接参数化：

$$\pi_{\theta}(a|s)$$

7.4.2 纯离散动作与纯连续动作

如果动作是有限个互斥选项，例如 `left / stay / right`，策略输出 categorical distribution（类别分布）：

$$\pi_{\theta}(a|s) = \text{softmax}(f_{\theta}(s))$$

网络给每个离散动作一个 logit，经 softmax 后得到概率，再从中采样一个动作。

如果动作是连续向量，例如机械臂三个关节的力矩：

$$a = (a_1, a_2, a_3)$$

策略常输出 Gaussian distribution（高斯分布）的参数：

$$\pi_{\theta}(a|s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma_{\theta}(s))$$

如果各维在给定状态下条件独立，可以输出每一维的均值和标准差，并把各维 log probability 相加：

$$\log \pi_{\theta}(a|s) = \sum_j \log \mathcal{N}(a_j; \mu_{\theta,j}(s), \sigma_{\theta,j}(s))$$

逐项解释：

- $\mathcal{N}$ ：花体大写 N ，表示 Normal Distribution（正态分布，也叫高斯分布）的概率密度函数。
- a_j ：本次实际采样并执行的第 j 维动作值； j 是动作维度索引。
- $\mu_{\theta,j}(s)$ ：策略网络在状态 s 下，为第 j 维动作预测的均值。
- $\sigma_{\theta,j}(s)$ ：策略网络预测的第 j 维标准差，表示这一维动作分布有多分散；实现中必须保证它大于 0。
- 分号 `;`：只是把“在哪里计算密度”与“分布参数”分开。 $\mathcal{N}(a_j; \mu_j, \sigma_j)$ 的意思是：均值为 μ_j 、标准差为 σ_j 的高斯分布，在 a_j 这个位置的密度值。
- $\sum_j$ ：把所有动作维度的 log probability 相加。

更完整地写，条件独立假设先给出联合概率密度：

$$\pi_{\theta}(a|s) = \prod_{j=1}^d \mathcal{N}(a_j; \mu_{\theta,j}(s), \sigma_{\theta,j}(s))$$

其中 d 是连续动作向量的维数。两边取对数后，乘积就变成求和：

$$\log \pi_{\theta}(a|s) = \log \prod_{j=1}^d p_j = \sum_{j=1}^d \log p_j$$

这里 $p_j = \mathcal{N}(a_j; \mu_{\theta_j}(s), \sigma_{\theta_j}(s))$ 。因此“各维 log probability 相加”并不是经验技巧，而是各维条件独立时联合密度乘法在对数域中的等价写法。

例如二维动作 $a = (\text{转向}, \text{油门}) = (0.2, 0.7)$ 。网络在状态 s 下输出：

$$\mu_{\theta}(s) = (0.1, 0.6), \sigma_{\theta}(s) = (0.2, 0.1)$$

那么这次动作的联合 log probability 是：

$$\log \pi_{\theta}(a|s) = \log \mathcal{N}(0.2; 0.1, 0.2) + \log \mathcal{N}(0.7; 0.6, 0.1)$$

注意：连续变量取到某个精确数值的概率为 0，所以 $\pi_{\theta}(a|s)$ 在这里严格来说是 probability density（概率密度），不是离散动作中的 probability mass（概率质量）。训练时使用它的 log density，通常仍简称 log probability 或 `log_prob`。

这只是常用参数化，不代表连续动作一定服从高斯，也不代表各维一定独立。策略分布应匹配动作约束和问题结构。

7.4.3 离散加连续的混合动作

很多任务不是“纯 categorical”或“纯 continuous”，而是一个动作同时包含：

$$A_t = (K_t, U_t)$$

其中：

- K_t 是 discrete action type（离散动作类型随机变量），例如选择技能、订单类型或控制模式。
- U_t 是 continuous action parameter（连续动作参数随机变量），例如瞄准方向、报价、力度或持续时间。
- 一次实际采样值写成 $a_t = (k_t, u_t)$ 。

这种空间常叫：

- hybrid action space：混合动作空间。
- parameterized action space：参数化动作空间，连续参数的含义由离散动作类型决定。

例子：

游戏动作：

k = 选择技能 {普通攻击, 位移, 大招}
 u = [瞄准角度, 释放距离]

交易动作：

k = {不操作, 买入, 卖出}
 u = [价格, 数量]

联合策略不能只写一个 categorical 或一个 Gaussian。常见分解是：

$$\pi_{\theta}(k, u|s) = \pi_{\theta_d}(k|s) \pi_{\theta_c}(u|s, k)$$

这里：

- $\pi_{\theta_d}(k|s)$ 是 categorical head，先选离散类型。
- $\pi_{\theta_c}(u|s, k)$ 是 conditional continuous head，在给定类型 k 后输出对应连续参数分布。
- θ_d / θ_c 分别表示离散分支和连续分支参数，也可以共享 backbone。

采样过程：

1. 输入状态 s 。
2. categorical head 采样离散类型 k 。
3. 读取类型 k 对应的 continuous distribution。
4. 从该分布采样连续参数 u 。
5. 把联合动作 (k, u) 交给环境。

联合动作的 log probability 是两部分相加：

$$\log \pi_{\theta}(k, u | s) = \log \pi_{\theta_d}(k | s) + \log \pi_{\theta_c}(u | s, k)$$

所以 vanilla REINFORCE 的一次策略 loss 可以直接写成：

$$L_t = - [\log \pi_{\theta_d}(k_t | s_t) + \log \pi_{\theta_c}(u_t | s_t, k_t)] G_t$$

同一个 G_t 会评价整个联合动作：既更新“离散类型选得对不对”，也更新“该类型下连续参数取得好不好”。

7.4.3.1 两种混合方式不能混淆

第一种是 同时独立输出多个动作分量。例如：

离散开关 k ：是否开启辅助控制
连续向量 u ：方向盘角度和油门

若建模上假设给定状态后两者独立，可以写：

$$\pi(k, u | s) = \pi_d(k | s) \pi_c(u | s)$$

第二种是 离散选择决定连续参数的含义。例如不同技能有不同参数范围，此时必须使用条件分布：

$$\pi(k, u | s) = \pi_d(k | s) \pi_c(u | s, k)$$

把第二类错误地写成完全独立，会让 continuous head 不知道自己正在为哪个离散动作产生参数。

7.4.3.2 只有被选中的连续分支参与更新

假设三个离散动作分别需要不同参数：

```
move(dx, dy)
shoot(angle, power)
wait()
```

采样到 shoot 时，只应计算 shoot 对应的 angle/power log probability。move 的 dx/dy 和 wait 的空参数在这一步没有执行，不应被当作已采样动作一起计入 loss。

常见实现是：

1. 网络为每个离散类型输出一组连续分布参数。
2. 用采样到的 k_t 选择对应分支。
3. 只对该分支的 u_t 计算 log probability。
4. 对 padding 或无效参数做 mask。

7.4.3.3 连续动作边界怎么处理

如果连续参数必须落在有限区间，例如力度在 $[0, 1]$ ，不能简单从无界 Gaussian 采样后任意截断，否则环境实际执行的动作与策略计算 log probability 的动作分布不一致。

常见做法：

- 用 `tanh` 把 Gaussian 样本压到 $[-1, 1]$ ，再缩放到合法范围。
- 计算 log probability 时加入 squashing transformation 的 Jacobian correction。
- 对不同离散类型使用各自的参数范围和 mask。

这些属于策略分布实现细节，但面试中至少要知道：**动作经过非线性变换后，log probability 也要对应变换后的真实分布。**

7.4.3.4 混合动作的主要难点

- 一个 return 同时评价离散选择和连续参数，credit assignment 更难。
- categorical 和 continuous 两部分的 entropy、梯度尺度可能差异很大。
- 某些离散动作很少被选中，其连续参数分支就缺少训练数据。
- 不同离散类型可能有不同维度、边界和合法性约束。
- 纯 DDPG/TD3 假设 actor 直接输出连续动作，纯离散 DQN 假设动作可枚举，都不能不加改造地处理这种联合动作。

本章只需要掌握联合策略和 REINFORCE loss。第 8 章会说明如何用 Critic 为联合动作提供更低方差的 advantage；第 10 章会说明连续控制算法处理混合动作时需要怎样拆分 Actor。

7.4.4 优化目标

一条轨迹写成：

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots)$$

策略目标是最大化轨迹回报的期望：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [G_{\theta}]$$

难点是：期望所依赖的轨迹分布 $p_{\theta}(\tau)$ 本身随 θ 改变。动作是采样出来的，不能把环境当作普通监督网络直接穿过动作反传。

7.5 Policy Gradient 是怎样推出来的

7.5.1 Log-derivative Trick

对任意可微概率分布：

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

因此：

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G_{\theta} d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) G_{\theta} d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G_{\theta} d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) G_{\theta}] \end{aligned}$$

这个变换的作用是：把“对采样分布求导”改写成“在该分布下采样，再对样本的 log probability 求导”。

7.5.2 为什么环境转移项会消失

轨迹概率可分解为：

$$p_{\theta}(\tau) = p(S_0) \prod_t \pi_{\theta}(A_t | S_t) P(S_{t+1} | S_t, A_t)$$

取对数后，乘积变成求和。初始状态分布和环境转移不依赖 θ ，所以：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

代回目标：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_0]$$

7.5.3 从整条轨迹回报到 Reward-to-go

A_t 不可能影响时刻 t 之前已经发生的奖励。把过去奖励也乘到当前动作的梯度上，不会增加有用信息，只会增加噪声。

因此可以把 G_0 换成从当前时刻开始的 reward-to-go：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

得到 REINFORCE 常用的梯度形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t]$$

这一步依赖因果关系：去掉的是当前动作无法影响的过去奖励，不是随意截断未来奖励。

7.5.4 为什么理论公式写 $Q_{\pi}(s, a)$

动作价值函数的定义正是：

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a]$$

把上一节公式对 (S_t, A_t) 做条件期望，就得到 Policy Gradient theorem 的常见形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S_t, A_t \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) Q_{\pi}(S_t, A_t)]$$

所以 Q_{π} 不是突然替代了 G_t ：

- G_t 是一条轨迹上实际采样到的随机回报。
- $Q_{\pi}(s, a)$ 是固定 (s, a) 后，对所有可能未来轨迹的 G_t 取期望。
- 单个 G_t 是 $Q_{\pi}(s, a)$ 的无偏 Monte Carlo 样本，但方差可能很大。

可以把关系写成：

$$Q_{\pi}(s, a) = \mathbb{E} [G_t | s, a]$$

理论推导喜欢写期望量 Q_{π} ；最基础的 REINFORCE 没有 Q 网络，只能用采样量 G_t 。

7.6 REINFORCE：用 G_t 估计 Q_{π}

7.6.1 单样本更新

REINFORCE 用一条或一批完整轨迹估计上一节的期望：

$$\theta \leftarrow \theta + \alpha_{\pi} \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

如果实现成最小化 loss：

$$L_{REINFORCE} = -\sum_t \log \pi_{\theta}(a_t | s_t) G_t$$

注意： G_t 在计算 policy loss 时被当作样本权重，不需要对环境奖励反向传播。

7.6.2 完整流程

```
initialize policy pi_theta
repeat:
    use pi_theta to sample complete trajectories
    for each time step t:
        compute Monte Carlo return G_t
    minimize -sum_t log pi_theta(a_t|s_t) * G_t
```

它属于：

- policy-based：直接更新 policy。
- on-policy：轨迹由当前要更新的策略采样。
- Monte Carlo：必须拿到后续完整奖励，才知道 G_t 。
- 不带 critic：没有额外价值网络做 bootstrap。

7.6.3 为什么方差很大

即使固定同一个 (s_t, a_t) ，后续仍可能因为环境随机性和之后采样的动作不同，得到差异很大的 G_t 。

例如同一动作的五次回报是：

2, 18, -4, 25, 9

这些值的平均数可以估计 $Q_{\pi}(s_t, a_t)$ ，但单次更新可能被偶然的 25 或 -4 强烈影响。Baseline 就是为这个高方差学习信号引入参考点。

7.7 Baseline：从哪里来，为什么有效

7.7.1 Baseline 是 Control Variate

Monte Carlo 估计中常用 control variate：从一个随机量中减去和它相关、但期望影响可控的参考量，以降低方差。

在 Policy Gradient 中，原始样本更新信号是：

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

我们希望把 G_t 改成“超过参考水平多少”：

$$G_t - b(s_t)$$

得到：

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - b(S_t))]$$

Baseline 不是环境给出的新 reward，也不是算法凭空知道的真值。它只是研究者选择或学习出的参考值。

7.7.2 为什么 Baseline 不能依赖当前动作

给定状态 s ，如果 $b(s)$ 不依赖当前动作：

$$\begin{aligned} & \mathbb{E}_{A \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(A | s) b(s)] \\ &= b(s) \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) \\ &= b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) \\ &= b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) \\ &= b(s) \nabla_{\theta} 1 = 0 \end{aligned}$$

因此减去 $b(s)$ 不改变梯度期望，只改变样本方差。

如果 baseline 直接依赖当前动作 a ，上面的 $b(s)$ 就不能提出求和，结果通常不再为零，可能改变真正要优化的梯度。

7.7.3 Baseline 可以怎样得到

Baseline 有三种常见来源：

1. **常数 baseline**：例如所有 episode return 的移动平均。
2. **按时间步 baseline**：例如 episodic 任务中，第 t 步历史平均 return。
3. **状态 baseline**：输入 s_t ，预测该状态通常能得到多少回报。

第三种最常用，因为不同状态的“正常分数”差异很大。状态很有利时，return 为 100 可能很普通；状态很糟时，return 为 20 反而可能很好。

7.7.4 为什么常用 $V_{\pi}(s)$

状态值的定义是：

$$V_{\pi}(s) = \mathbb{E}_{A \sim \pi_{\theta}(\cdot | s)} [Q_{\pi}(s, A)] = \mathbb{E}_{\pi} [G_t | S_t = s]$$

它正好表示“在状态 s 按当前策略行动时，平均能得到多少回报”，因此是自然的状态 baseline：

$$b(s) = V_{\pi}(s)$$

但 $V_{\pi}(s)$ 通常未知。REINFORCE with baseline 可以训练一个函数 $b_{\phi}(s)$ ，用采样到的 Monte Carlo return 做回归：

$$L_b(\phi) = \mathbb{E}_t [(b_{\phi}(s_t) - G_t)^2]$$

这里 ϕ 是 baseline 模型参数。它的监督标签仍然是完整 G_t ，没有 bootstrap。

7.7.5 Baseline 是否一定降低方差

“任意 action-independent baseline 都不引入 bias”是期望层面的结论，但它不代表任意 baseline 都一定降低方差。

- Baseline 接近该状态下的典型回报时，通常能显著降低方差。
- Baseline 很差或尺度异常时，也可能让方差更大。
- 严格的最小方差 baseline 还会受到 score gradient 大小的加权影响，不一定精确等于 $V_{\pi}(s)$ 。

实践中选择 $V_{\pi}(s)$ ，是因为它含义清楚、容易学习，而且通常是很好的近似。

7.7.6 REINFORCE with Baseline 的完整流程

```

initialize policy pi_theta
initialize baseline b_phi
repeat:
    sample complete trajectories using pi_theta
    compute every Monte Carlo return G_t
    compute A_hat_t_MC = G_t - b_phi(s_t)

    update theta with:
        -log pi_theta(a_t|s_t) * stop_gradient(A_hat_t_MC)

    update phi with:
        (b_phi(s_t) - G_t)^2
  
```

这里虽然有两个可学习模块，但 policy 学习信号仍依赖完整 Monte Carlo return。它通常称为 **REINFORCE with baseline**。第 8 章会用 bootstrap 和 TD 方法把 baseline estimator 变成真正的 Critic。

7.8 Advantage：为什么从 Baseline 自然出现

7.8.1 理论 Advantage

如果在理论 Policy Gradient 中选择：

$$b(s) = V_{\pi}(s)$$

那么：

$$\begin{aligned} Q_{\pi}(s, a) - b(s) &= Q_{\pi}(s, a) - V_{\pi}(s) \\ &= A_{\pi}(s, a) \end{aligned}$$

因此 Policy Gradient theorem 也可以写成：

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) A_{\pi}(S_t, A_t)]$$

理论 advantage 的含义是：

在状态 s 下先做动作 a ，比按照当前策略随机选择一个动作平均好多少。

7.8.2 实现中并没有直接得到真实 Q_{π}

REINFORCE 不知道真实 Q_{π} 和 V_{π} 。它只有：

- 一次采样到的 G_t ，用来估计 $Q_{\pi}(s_t, a_t)$ 。
- 学到的 $b_{\phi}(s_t)$ ，用来估计 $V_{\pi}(s_t)$ 。

所以实际使用的是：

$$\hat{A}_t^{MC} = G_t - b_{\phi}(s_t)$$

对应关系必须分清：

层次	动作价值	状态基线	Advantage
理论期望	$Q_{\pi}(s, a_t)$	$V_{\pi}(s_t)$	$A_{\pi} = Q_{\pi} - V_{\pi}$
REINFORCE 样本估计	G_t	$b_{\phi}(s_t)$	$\hat{A}_t^{MC} = G_t - b_{\phi}(s_t)$

所以不是“advantage 不用 G_t ，直接换成了 $Q(s, a)$ ”。正确说法是：

理论公式把未来随机回报的条件期望记作 Q_{π} ；REINFORCE 用单条轨迹的 G_t 估计它。减去状态 baseline 后， $G_t - b_{\phi}(s_t)$ 是 theoretical advantage 的 Monte Carlo 估计。

7.8.3 数值例子

在状态 s 下，当前策略选择三个动作的概率和真实动作价值为：

动作	$\pi(a s)$	$Q_{\pi}(s, a)$
left	0.2	12
stay	0.5	8
right	0.3	4

状态价值是按当前策略的平均动作价值：

$$V_{\pi}(s) = 0.2 \times 12 + 0.5 \times 8 + 0.3 \times 4 = 7.6$$

三个动作的 theoretical advantage：

$$\begin{aligned} A_{\pi}(s, \text{left}) &= 12 - 7.6 = 4.4 \\ A_{\pi}(s, \text{stay}) &= 8 - 7.6 = 0.4 \\ A_{\pi}(s, \text{right}) &= 4 - 7.6 = -3.6 \end{aligned}$$

如果本次采样了 **left**，实际 $G_t = 15$ ，baseline 预测 $b_{\phi}(s) = 7$ ，那么用于这次更新的是：

$$\hat{A}_t^{MC} = 15 - 7 = 8$$

这个 8 不是理论 $A_{\pi}(s, \text{left}) = 4.4$ ，而是一次带噪声的样本估计。多次采样平均后，才会接近理论值。

7.8.4 与 Actor-Critic 的分界

两者都可能写成：

$$-\log \pi_{\theta}(a_t | s_t) \hat{A}_t$$

区别不在这行 policy loss 的外观，而在 $\hat{A}_t$ 怎样得到：

方法	Advantage estimate 来源	是否 bootstrap	何时能更新
REINFORCE	G_t	否	通常等完整 episode
REINFORCE with baseline	$G_t - b_{\phi}(s_t)$	否	通常等完整 episode
Actor-Critic	TD error、n-step 或 GAE	通常是	可以较短 rollout 后更新

有些资料会把“任何带可学习 value baseline 的策略梯度”宽泛称为 Actor-Critic。为了学习脉络清楚，本课程用更严格的分界：

- 只用完整 G_t 回归 baseline，并用 $G_t - b_\phi$ 更新策略：仍放在 REINFORCE。
- Critic 开始用 Bellman bootstrap、TD 或 n-step target 估值：进入第 8 章 Actor-Critic。

7.9 两个完整例子

7.9.1 例子一：为什么不能只看 Return 的正负

假设一个状态本来就很难，当前策略的平均 return 是 -100。某次动作后得到 -60。

- 不加 baseline： $G_t = -60$ ，看起来应该降低该动作概率。
- 使用 $b(s_t) = -100$ ： $G_t - b(s_t) = 40$ ，说明它比这个状态的平均表现更好，应该提高概率。

Baseline 把“绝对分数”变成“相对当前状态预期的分数”。

7.9.2 例子二：Softmax 策略怎样被更新

两个动作当前概率为：

动作	当前概率	$\hat{A}_t^{MC}$
left	0.4	+3
right	0.6	-2

策略 loss 为：

$$L = -\log \pi_\theta(a_t | s_t) \hat{A}_t^{MC}$$

- 采到 left 且 advantage 为正：最小化 loss 会提高 $\log \pi(\text{left} | s)$ ，从而提高该动作概率。
- 采到 right 且 advantage 为负：最小化 loss 会降低该动作概率。

策略不是直接把概率加减一个固定值，而是通过 log probability 的梯度更新网络参数；softmax 会自动保证所有动作概率仍然和为 1。

7.10 本章面试高频问题

7.10.1 Policy Gradient theorem 里为什么是 $Q_\pi(s, a)$?

因为 $Q_\pi(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$ ，它表示当前动作之后未来回报的条件期望。Policy Gradient 用它衡量提高该动作概率对期望 return 的影响。

7.10.2 REINFORCE 为什么又使用 G_t ?

真实 Q_π 未知。REINFORCE 采样完整轨迹，用一次真实的 reward-to-go G_t 作为 $Q_\pi(s_t, a_t)$ 的无偏 Monte Carlo 样本。

7.10.3 Baseline 从哪里来?

它来自 control variate 的降方差思想。Baseline 可以是常数、历史平均值或按状态预测的回报。最常见的是训练 $b_\phi(s)$ 回归 Monte Carlo return，用它近似 $V_\pi(s)$ 。

7.10.4 为什么 Baseline 不引入 Bias?

只要 baseline 不依赖当前动作：

$$\mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) b(s)] = 0$$

所以它不会改变梯度期望。

7.10.5 Advantage 和 $G_t - b(s_t)$ 是同一个东西吗？

不完全是。 $A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$ 是理论期望量； $G_t - b_{\phi}(s_t)$ 是用一条轨迹和一个估计 baseline 得到的 Monte Carlo advantage estimate。

7.10.6 REINFORCE with Baseline 和 Actor-Critic 的区别？

前者仍用完整 G_t 训练 baseline、更新 policy，不 bootstrap；Actor-Critic 的 critic 通常使用 TD、n-step 或其他 bootstrap target，可以用更短 rollout 更新，但会引入 critic estimation bias。

7.11 本章练习

1. 从 $J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [G_0]$ 推导到 trajectory-level Policy Gradient。
2. 解释为什么 G_t 可以作为 $Q_{\pi}(s_t, a_t)$ 的样本，但二者不是同一个数学对象。
3. 证明 action-independent baseline 不改变 Policy Gradient 的期望。
4. 说明 baseline 为什么常选 $V_{\pi}(s)$ ，以及 $V_{\pi}(s)$ 未知时如何得到近似。
5. 某状态下两动作概率分别为 0.25 和 0.75，动作价值分别为 10 和 2。计算 $V_{\pi}(s)$ 和两个 advantage。
6. 对比 REINFORCE、REINFORCE with baseline 和 Actor-Critic 的学习信号。

▼ 参考答案（点击展开）

1. **Trajectory-level 推导：** 先把期望写成积分，得到 $\nabla_{\theta} J = \int \nabla_{\theta} p_{\theta}(\tau) G_0 d\tau$ ；使用 $\nabla p = p \nabla \log p$ 变成 $\mathbb{E} [\nabla_{\theta} \log p_{\theta}(\tau) G_0]$ 。再展开 $p_{\theta}(\tau) = p(S_0) \prod_t \pi_{\theta}(A_t | S_t) P(S_{t+1} | S_t, A_t)$ 。初始状态和环境转移不依赖 θ ，因此只留下 $\sum_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 。
2. **G_t 与 Q_{π} ：** G_t 是一条具体轨迹从 t 开始实际得到的随机累计奖励； $Q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$ 是对所有可能后续轨迹取条件期望。固定 (s, a) 反复采样， G_t 的样本均值才会接近 $Q_{\pi}(s, a)$ 。
3. **Baseline 证明：** 给定 s ，若 $b(s)$ 不依赖动作，则 $\sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) b(s) = b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a|s) = b(s) \nabla_{\theta} 1 = 0$ 。因此减去 baseline 不改变期望梯度。
4. **为什么选 $V_{\pi}(s)$ ：** $V_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$ 表示该状态按当前策略行动的平均回报，正好是判断动作相对好坏的参考水平。真实 V_{π} 未知时，可训练 $b_{\phi}(s)$ 最小化 $(b_{\phi}(s) - G_t)^2$ ，用 Monte Carlo return 做监督标签。
5. **数值题：** $V_{\pi}(s) = 0.25 \times 10 + 0.75 \times 2 = 4$ 。第一个动作的 advantage 为 $10 - 4 = 6$ ，第二个动作的 advantage 为 $2 - 4 = -2$ 。按策略加权后的 advantage 均值为 $0.25 \times 6 + 0.75 \times (-2) = 0$ 。
6. **三者对比：** REINFORCE 直接使用 G_t ；REINFORCE with baseline 使用 $G_t - b_{\phi}(s_t)$ ，两者都需要完整 Monte Carlo return。Actor-Critic 用 Critic 构造 TD、n-step 或 GAE advantage，通常会 bootstrap，可以更早更新且方差较低，但会引入 Critic 的估计偏差。